A smaller presentation of intuitionistic primitive recursive arithmetic

First-instance witness discharge

Research report prepared for Vladimir Reshetnikov

11 November 2026

Abstract

The current witness-discharge presentation of full first-order iPRA has three inference families. Its elimination rule takes a major formula, a local equality proof $b = t$, and a local formula proof F, then exports the normalized instance $F[t/b]$. We prove that the equality proof is unnecessary as a primitive premise: a deterministic test of the opened clause's zero and first instances supplies the only substitution that the rule needs to perform. The new rule has two proof premises and no replacement-term argument. Nevertheless it preserves every open sequent, not merely the closed equations.

The reconstruction obtains ordinary elimination and numerical cases before equality, and then recovers arbitrary transport through a two-row naming matrix. The inherited full-iPRA bi-interpretation therefore has both numerical domains whole, actual equality, and identity numerical composites. We include the complete primitive specification, proofs, the normalization and arithmetic bridges, a standalone Python checker, proof translations, adversarial examples, and reproducible measurements. The result is a concrete simplification relative to the current witness-discharge presentation, not an absolute minimality theorem.

Contents

1 Result and comparison boundary **3**

2 Discharging a witness using its first clause instance **4**
- 2.1 Complete native specification . . . 4
- 2.2 Soundness and the escape condition . . . 5
- 2.3 Ordinary elimination and numerical cases without equality . . . 6
- 2.4 Recovering equality from the fixed first instance . . . 6
- 2.5 Exact equivalence with unrestricted witness discharge . . . 8
- 2.6 Implementation invariants and claim boundaries . . . 8

3 Normalization and a direct bridge to raw Choice-DI **9**
- 3.1 Strong normalization of scalar computation . . . 9
- 3.2 Confluence and normalization–substitution coherence . . . 10
- 3.3 The raw reference calculus and its harmless clause convention . . . 11
- 3.4 Raw provability of normalization, including update binders . . . 12
- 3.5 Repairing the omitted case introduction at witness zero . . . 13
- 3.6 An auxiliary normal-form calculus . . . 13
- 3.7 Both proof translations and both composites . . . 14
- 3.8 Uniform normalization is external . . . 15

4 Complete checker, certificate syntax, and measurements **15**
4.1 Numerical and formula codes . . . 15
4.2 A uniform three-field proof packet . . . 15
4.3 The accepted-trace invariant . . . 16
4.4 Measured comparisons and their interpretation . . . 16
4.5 Executed evidence . . . 17

5 A primitive-recursive certificate interface **17**

Attribution and role of the inherited foundations **20**

A Complementary introductions with one local assumption **20**
A.1 Complete native syntax and rule inventory . . . 20
A.2 Exact equivalence and the unused-witness construction . . . 21
A.3 Induction and numerical cases separate cleanly . . . 22
A.4 The entire ordinary logical interface . . . 23
A.5 Whole-domain realizations and both formula composites . . . 24
A.6 What the theorem does and does not claim . . . 25

B Arithmetic strength and both ordinary interpretation composites **25**
B.1 The arithmetic dependency order . . . 26

C Internal scalar predecessor bootstrap **27**
C.1 Elementary terms and two finite cycles . . . 27
C.2 A ramp with a global scaling identity . . . 28
C.3 A comparator from scalar iteration . . . 28
C.4 Saturated iteration gives predecessor . . . 29
C.5 Dependency boundary . . . 30

D From literal-equation induction to full intuitionistic PRA **30**
D.1 The exact hypotheses . . . 31
D.2 Constructor separation and two elementary tests . . . 31
D.3 Four elementary uses of equation induction . . . 32
D.4 An equation invariant that reflects actual equality . . . 33
D.5 Compilation of all quantifier-free formulas . . . 34
D.6 Recovery of induction and the full theory . . . 35

E Pairing, primitive recursion, and internal numerical composites **35**
E.1 Pairing from a bounded square-root search . . . 36
E.2 Compiling each primitive recursion . . . 36
E.3 Why both numerical round trips are internal . . . 37

F Complete normalized acceptance checker **38**

G Reproduction and file map **39**

1 Result and comparison boundary

The comparison repository is ProveIt [1], under the subtree

`Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`.

The pinned commit is

`17898ea7e6a1300dfdcad71040d5daeed67916d4`.

Its README lists sixty-seven filed research packages. We also inspected the six archives in `docs/incoming`, including the recent `witness_discharge` and `uniform_binding` packages. The immediate mathematical control is the incoming `ipra_witness_discharge_2026-11-11.zip` [2]. In particular, three inference families and an initial reflexivity clause are already present there. The new theorem concerns deletion of one *proof premise* and deterministic recovery of its substitution.

The complete proposed calculus is denoted $\mathcal{A}$. Its formulas have only equality and the paired binder D; its numerical terms are canonical scalar iterators. There is one initial clause and three inference families: Use, Intro, and Name. The standard reading is

$$D_{x,y}B(x,y) \quad \rightsquigarrow \quad \exists x\big(B(x,0) \to \forall y\, B(x,y)\big).$$

The syntactic reductions used to obtain canonical terms are explicitly part of the theory's specification and of the measured checker.

Reference theory. Here iPRA means full intuitionistic first-order logic with equality, zero, successor, and a function symbol for every fixed primitive-recursive description, the usual defining equations and successor axioms, and induction restricted to quantifier-free formulas, with parameters. It is not the quantifier-free proof language alone. The equivalent atomic-induction convention in the full primitive-recursive language is also used in the literature [8]; the equality-reflection appendix gives the required equivalence in this development.

What is proved. For every canonical open clause sequent, the old witness-discharge calculus $\mathcal{W}$ and $\mathcal{A}$ prove exactly the same conclusion from exactly the same assumptions. The translation is the identity on their numerical and formula languages. Composing with the scalar arithmetic bridge proves full bi-interpretability and equiconsistency with iPRA. The two numerical composite interpretations are internally isomorphic through the graph $x = y$; both formula composites are also proved, so this is stronger than a standard-model agreement or an equational conservativity statement.

What is inherited. The scalar iterator, the D binder, the first-order logical macros, full computation normalization, the two-row naming method, predecessor, equality reflection, and the primitive-recursive compiler are inherited from the repository. The new contribution is the completeness of a fixed first-instance restriction on witness discharge. The transparent certificate representation and its complete implementation are also given, with matched controls. No universal priority or absolute minimum-source-size claim is made.

How to read the report. Section 2 contains the complete native axiomatization and the new proof. Section 3 connects canonical syntax to the raw scalar basis. The implementation section records the complete checker contract and its measured scope. The appendices reproduce the attributed logical and arithmetic foundations, so the full-iPRA conclusion does not rest on an omitted predecessor or pairing argument.

2 Discharging a witness using its first clause instance

The incoming witness-discharge presentation has a separate proof of an equality $b = t$ inside each eliminated witness scope. That premise is unnecessary. It suffices to recognize an equality in one fixed instance of the clause which the witness itself supplies. If that instance has no usable equality, the witness is discharged without substitution. The choice is determined by the major formula; the certificate supplies neither an equality proof nor a replacement term.

This is a restriction of the previous inference, with a nontrivial completeness theorem. It does not rely on the numerical standard model, semantic truth testing, or an unlisted equality rule.

2.1 Complete native specification

Use one numerical sort. Let N denote full computation normalization of the scalar terms

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h)$$

for the context-closed reductions

$$I(0, a; u.h) \longrightarrow a, \qquad I(Sn, a; u.h) \longrightarrow h[I(n, a; u.h)/u]. \tag{NF}$$

Only the state variable u in h is bound. Every displayed term means its normal form, and every substitution is capture-avoiding and followed by N. Normalization of a formula acts on numerical terms and leaves its formula constructors intact. We use the established termination, confluence, and substitution-coherence results for these reductions. The normalization algorithm is part of the trusted syntax and must be included in a complete-checker measurement.

Formulas are

$$A ::= (s = t) \mid D_{x,y}A,$$

where x, y are distinct bound numerical variables. A reusable context clause is $[\vec{A}; y.B]$, where $\vec{A}$ has zero or one antecedent, and y is absent from every antecedent. Its ordinary reading is

$$[\,; y.B]^\circ = \forall y B^\circ, \qquad [A; y.B]^\circ = A^\circ \to \forall y B^\circ.$$

An ordinary assumption A abbreviates $[\,; v.A]$, with v unused. Contexts are finite lists of clause occurrences. Every proof starts with the single explicit initial clause

$$\mathcal{R} = [\,; y.y = y]. \tag{Ax}$$

Put $H = (0 = 0)$, $1 = S0$, and $2 = S1$. The designated contradiction is $0 = 1$. There are exactly the following three primitive inference families.

Use. For a retained $[\vec{A}; y.B]$ and any well-scoped term t, infer $N(B[t/y])$ after proving the antecedent if there is one. There is no proof premise for a clause with no antecedent.

$$\frac{\{\Gamma \vdash A : A \text{ is the sole antecedent, if present}\}}{\Gamma \vdash N(B[t/y])}. \tag{Use}$$

Intro. For fresh z and a well-scoped witness t,

$$\frac{\Gamma, N(B(t, \eta)) \vdash N(B(t, Sz))}{\Gamma \vdash D_{x,y}B(x, y)}, \qquad \eta = \begin{cases} z, & B \text{ is an equation and } t \text{ is literally } 0, \\ 0, & \text{otherwise.} \end{cases} \tag{Intro}$$

Here z is fresh for Γ, t and all fixed parameters of B. The displayed ordinary assumption is discharged.

Name. After proving $D_{x,y}B$, take a fresh numerical parameter b and form

$$B_b(y) = N(B(b,y)), \qquad A_0 = N(B_b(0)), \qquad A_1 = N(B_b(1)), \qquad \mathcal{B}_b = [A_0; y.B_b(y)].$$

Define the replacement term *deterministically* by

$$\tau_b(B) = \begin{cases} t, & A_0 \text{ is literally } H \text{ and } A_1 \text{ is literally the equation } b = t, \\ b, & \text{otherwise.} \end{cases} \qquad \text{(First-instance test)}$$

The literal equation test is ordered: a formula $t = b$ is not recognized unless its left side is itself literally b. The term t is the right side of A_1, so no separate term argument is supplied. Then the rule is

$$\frac{\Gamma \vdash D_{x,y}B(x,y) \qquad \Gamma, \mathcal{B}_b \vdash F}{\Gamma \vdash N(F[\tau_b(B)/b])}, \qquad \text{(Name)}$$

provided the exported normal form has no free b. The parameter b is fresh for Γ and every fixed parameter of B. The branch formula F is well scoped in the extended numerical context. A recognized t may contain b; substitution is simultaneous, so an inserted copy of t is not substituted again.

These are all the primitive inferences. Reflexivity is a Use of $\mathcal{R}$. Computation is accounted for by canonical syntax. There are no additional primitive equality, logical, induction, case, cut, or weakening rules.

2.2 Soundness and the escape condition

The ordinary formula reading is

$$(s = t)^\circ = (s = t), \qquad (D_{x,y}B)^\circ = \exists x(B^\circ(x,0) \to \forall y B^\circ(x,y)).$$

For this subsection, use intuitionistic first-order equality logic with the scalar computation equations, numerical coverage, separation $0 = 1 \to \bot$, and contextual induction for literal equations. This is the inherited scalar theory with a full-iPRA bi-interpretation.

Lemma 2.1 (The replacement equality is always available)**.** *In the context $\Gamma, \mathcal{B}_b$, one can derive $b = \tau_b(B)$ using only the initial reflexivity clause and Use.*

Proof. If the test fails, $\tau_b(B) = b$ and Use of $\mathcal{R}$ at b proves $b = b$. If it succeeds, the antecedent of $\mathcal{B}_b$ is exactly H. Use of $\mathcal{R}$ at zero proves H, and Use of $\mathcal{B}_b$ at 1 proves its normalized first instance, which is exactly $b = \tau_b(B)$. No equality reasoning occurs in either derivation. □

Proposition 2.2 (Soundness of Name)**.** *Rule* (Name) *is sound for the ordinary reading, including when the recognized replacement term contains b.*

Proof. Ordinary existential elimination opens the major formula at b with the assumption represented by $\mathcal{B}_b$. The branch gives F. Lemma 2.1 gives $b = \tau_b(B)$, and ordinary equality substitution gives $F[\tau_b(B)/b]$. Computation equations identify this with the exported normal form. That normal form contains no free b, so ordinary existential elimination discharges the witness and its clause. Equality substitution itself does not require b to be absent from its right side; only existential discharge needs the stipulated absence from the final exported formula. □

Use has its displayed ordinary reading. Intro is sound by numerical coverage in case mode and literal-equation induction in progress mode. Thus the entire presentation is sound in the inherited scalar theory.

2.3 Ordinary elimination and numerical cases without equality

Lemma 2.3 (Ordinary paired elimination)**.** *The usual rule*

$$\frac{\Gamma \vdash D_{x,y}B \qquad \Gamma, \mathcal{B}_b \vdash A}{\Gamma \vdash A}, \qquad b \notin \mathrm{FV}(A),$$

is derivable with one use of Name.

Proof. Apply Name with its supplied branch formula A. Since A is a normal formula with no free b, substituting any term for b leaves it unchanged. Thus $N(A[\tau_b(B)/b]) = A$ regardless of whether the first-instance test succeeds. There is no need to force the test to fail or to choose a target. □

Weakening, injective fresh renaming, and ordinary-assumption cut are admissible by induction on proof trees. Freshen local eigenparameters before inserting proofs. The deterministic test depends only on the major formula and is unchanged by an injective renaming. An ordinary assumption is an unconditional constant clause, so its Use nodes can be replaced by the supplied proof. None of these structural constructions uses general equality transport or arbitrary numerical substitution of proof trees.

For unused u, write $q_y A(y) = D_{u,y}A(y)$.

Lemma 2.4 (Conditional closure and numerical cases)**.** *The calculus derives*

$$\frac{\Gamma, A(0) \vdash A(Sz)}{\Gamma \vdash q_y A(y)}, \qquad \frac{\Gamma \vdash q_y A(y) \qquad \Gamma \vdash A(0)}{\Gamma \vdash A(t)},$$

and consequently

$$\frac{\Gamma \vdash A(0) \qquad \Gamma \vdash A(Sz)}{\Gamma \vdash A(t)} \tag{Cases}$$

for every native formula A and appropriate fresh z. These constructions use no equality transport or equation induction.

Proof. For the first rule, introduce the unused witness at the numeral 1. It is not literally zero, so Intro uses the case premise at every matrix root. For the second, use Lemma 2.3. The opened clause is $[A(0); y.A(y)]$, because the witness coordinate is unused. Supply its antecedent and apply Use at t. The result has no free existential witness, so ordinary elimination closes. For Cases, first weaken the successor proof by the unused ordinary assumption $A(0)$, then combine the preceding two constructions. □

2.4 Recovering equality from the fixed first instance

Define normalizing selector and parity terms by

$$C(v, a, c) = N(I(v, a; u.c)), \qquad u \notin \mathrm{FV}(c), \qquad P(y) = N(I(y, 0; u.C(u, 1, 0))).$$

Direct computation gives literal identities of canonical syntax:

$$C(0, a, c) \equiv a, \qquad C(Sv, a, c) \equiv c, \qquad P(1) \equiv 1, \qquad P(2) \equiv 0. \tag{Endpoints}$$

Only these fixed endpoints of P are used; no parity induction or range theorem is required.

Theorem 2.5 (Uncrossed term transport)**.** *For every fixed numerical template $h(w)$, the rule*

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash N(h(s)) = r}{\Gamma \vdash N(h(t)) = r}$$

is derivable. It uses no new arithmetic assumption and no supplied equality branch at Name.

Proof. Choose x, y fresh for all passive data. Let $h(x)$ denote its canonical instance and form the equation matrix

$$B(x,y) = \big(C(y,0,C(P(y),N(h(x)),x)) \\ = C(y,0,C(P(y),r,t))\big). \qquad \text{(Naming matrix)}$$

Introduce $D_{x,y}B(x,y)$ at witness s. Its successor instance is

$$C(P(Sz),N(h(s)),s) = C(P(Sz),r,t).$$

Apply Cases to the template

$$A(v) = \big(C(v,N(h(s)),s) = C(v,r,t)\big)$$

at target $P(Sz)$. Its zero instance is exactly the supplied equation $N(h(s)) = r$, and its successor instance is exactly the supplied equation $s = t$. The eigenparameter used for this Cases argument is fresh for the current local context and is different from z.

The resulting successor proof ignores the ordinary assumption of Intro. Consequently it is valid whether s is literally zero, selecting progress, or any other term, selecting cases. This constructs the major formula without any equality substitution.

Open that major at fresh b. By the endpoint computations,

$$B(b,0) \equiv H, \qquad B(b,1) \equiv (b = t), \qquad B(b,2) \equiv (N(h(b)) = r).$$

The first-instance test therefore selects exactly $\tau_b(B) = t$. For the sole branch of Name, prove H by the fixed reflexivity clause and specialize the opened clause at 2. This yields $F = (N(h(b)) = r)$. Name exports

$$N\big((N(h(b)) = r)[t/b]\big) \equiv (N(h(t)) = r)$$

by normalization–substitution coherence. The parameter b was fresh for h, t, r, so the exported formula is outer scoped.

The premise carrying $s = t$ is used while proving the major naming formula, not as a third premise of Name. Thus the proof does not smuggle the deleted equality branch back into the rule's checking interface. □

Corollary 2.6 (All equality reasoning)**.** *Symmetry, transitivity, congruence, crossed transport, and transport in every native formula template are derivable.*

Proof. For symmetry, take $h(w) = w$ and $r = s$ in Theorem 2.5; the minor premise is reflexivity $s = s$. This proves $t = s$ from $s = t$. Uncrossed transport followed by symmetry gives crossed transport. A reflexive minor gives congruence, followed by symmetry if necessary. Symmetry of $a = b$ and transport of $b = c$ through the identity template give $a = c$.

For transport in an equation $p(w) = q(w)$, transport its left side, reverse the resulting equation, transport its right side, and reverse again. This proves $p(t) = q(t)$ from $s = t$ and $p(s) = q(s)$.

For a compound template $D_{x,y}E(w,x,y)$, argue by external induction on formula depth. Open $D_{x,y}E(s,x,y)$ at a fresh c using ordinary elimination. Introduce $D_{x,y}E(t,x,y)$ at witness c. Since c is a fresh parameter, its syntax is not literally zero, and Intro selects cases; no proposition $c \neq 0$ is assumed. From the temporary base $E(t,c,0)$, use the induction hypothesis and symmetry to obtain $E(s,c,0)$. Use the opened old clause at Sz to obtain $E(s,c,Sz)$, then transport forward by the induction hypothesis to $E(t,c,Sz)$. Close Intro and ordinary elimination. Every recursively transported matrix has smaller formula depth. This is not induction over quantified formulas inside the arithmetic theory. □

2.5 Exact equivalence with unrestricted witness discharge

Let $\mathcal{W}$ be the incoming canonical witness-discharge calculus: the same terms, formulas, clauses, fixed axiom and Use/Intro rules, with

$$\frac{\Gamma \vdash D_{x,y}B \qquad \Gamma, \mathcal{B}_b \vdash b = t \qquad \Gamma, \mathcal{B}_b \vdash F}{\Gamma \vdash N(F[t/b])} \tag{Old Name}$$

and the same freshness and export conditions. Let $\mathcal{A}$ denote the deterministic calculus just specified.

Theorem 2.7 (Identity-language sequent equivalence)**.** *For every finite canonical clause context* Γ *containing the distinguished reflexivity axiom and every well-scoped canonical formula* A*,*

$$\Gamma \vdash_{\mathcal{A}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{W}} A.$$

The two translations are effective transformations of finite derivations and preserve the complete end sequent literally.

Proof. From $\mathcal{A}$ to $\mathcal{W}$, translate the two premises of each new Name recursively and supply the missing equality proof from Lemma 2.1. All other rules are shared.

In the converse direction, recursively translate the three premises of an Old Name inference. In its extended witness context, use Corollary 2.6 to transport the translated branch formula along the translated equation $b = t$. This gives the prescribed normal form $N(F[t/b])$. Its original export condition says that it has no free b, so Lemma 2.3 closes the major existential. This works even when t contains b, because the same ordinary equality substitution and final export condition apply.

The apparent dependency is not circular: ordinary elimination and Cases were obtained before any equality transport; the naming-matrix proof of term transport uses only those constructions, the supplied premises, and one new Name. Formula transport then uses strictly smaller matrix depth. Every replacement is therefore a finite, explicitly defined proof construction. □

Corollary 2.8 (Full iPRA, not just the equational fragment)**.** *The ordinary realization of* $\mathcal{A}$ *is bi-interpretable and equiconsistent with full first-order intuitionistic primitive recursive arithmetic. Both numerical interpretation domains are whole, equality is actual equality, and both numerical composite maps have graph* $x = y$*.*

Proof. Compose Theorem 2.7 with the incoming canonical calculus equivalence and its inherited scalar arithmetic and logical bi-interpretation. The present translation is the identity on the entire native language and every end sequent, so it preserves both inherited formula round trips and the numerical identity maps. In particular it preserves the designated contradiction $0 = 1$, yielding equiconsistency. The scalar compiler, equality reflection, predecessor, logical macros, and arithmetic bridge are inherited and must be included or precisely referenced; the local result does not replace their proofs. □

2.6 Implementation invariants and claim boundaries

A forward checker may infer the major formula and the branch formula. At Name it checks that the major has a D root, opens its matrix at the newest fresh parameter, computes its zero and first instances, and determines $\tau_b(B)$ by (First-instance test). It then checks the branch under the one new clause, performs simultaneous free-parameter substitution, normalizes, and verifies that the result has no free b. The original outer goal is compared with this inferred result.

Scope validation is essential even if repeated formula-grammar checks are removed: a branch may use the fresh witness in an equation, but Name may export only an outer-scoped formula. The clause antecedent must be the actual normalized zero instance of the same opened matrix;

replacing it by a claimed true formula would be unsound. The recognized first instance must be an equation with left side literally the current witness. Merely having an equation root, being provably equal to that shape, or having the witness on the right does not authorize the substitution in this rule.

There is a useful implementation invariant: if the computed zero instance is literally H, then the matrix has an equation root, and so does its first instance. Normalization never deletes a D constructor. A checker which has already validated the major formula may therefore omit a second equation-root test on the first instance; the test $A_0 = H$ already ensures that its two equation fields exist. This is a structural fact about the validated grammar, not a relaxation of the mathematical guard.

Numerical substitution can change the selected Intro mode or create the first-instance guard. A primitive-tag-preserving substitution theorem is not claimed. Admissibility follows through the exact equivalence with the reference calculus and its established substitution theorem or repairs.

The simplification deletes one proof premise and two certificate payloads (the replacement term and, for a synthesizing checker, the claimed branch formula) from the earlier interface. It still has three inference families and one explicit reflexivity clause. A measured checker reduction must count the deterministic guard and normalization in full. It is neither a proof of absolute minimum size nor a claim that three families first occurred here. The new mathematical assertion is the exact completeness of the first-instance restriction.

Uniform full normalization over arbitrary finite iterator descriptions is total recursive but need not be primitive recursive. A finite budget or explicit reduction certificates can give a primitive-recursive bounded certificate predicate without changing the provable formulas; that interface is distinct and its code must also be counted. No object-level uniform evaluator or new totality axiom is asserted by this metasyntactic operation.

3 Normalization and a direct bridge to raw Choice-DI

This section reconstructs the normalization and raw-comparison arguments of the repository's full-normalization package [3] and the incoming witness-discharge report [2]. The compound-envelope repair is inherited from those sources. The direct comparison below connects the present deterministic Name calculus to the raw Choice-DI foundations included later, without taking the incoming witness-discharge equivalence as a black box. The explicit administrative-`let` argument also accounts for the selected normalizer's strict evaluation of a recursive value that its update may subsequently discard.

All assertions about normalization in this section are external theorems about finite syntax. They are not new arithmetic axioms or an object-language uniform evaluator.

3.1 Strong normalization of scalar computation

On raw terms $x, 0, St, I(n, a; u.h)$ let $\longrightarrow$ be the contextual closure, including under update binders, of

$$I(0, a; u.h) \longrightarrow a, \qquad I(Sn, a; u.h) \longrightarrow h[I(n, a; u.h)/u]. \tag{Raw computation}$$

Bound variables are freshened before substitution. These reductions are stable under capture-avoiding substitution. Call a finite term strongly normalizing if it has no infinite reduction sequence. For such a term t, write $\nu(t)$ for the largest length of a reduction beginning at t and $|t|$ for its syntax-tree size. The maximum exists because the reduction tree is finitely branching and has no infinite path.

Lemma 3.1 (Closure under iteration)**.** *Suppose n, a are strongly normalizing and $h[v/u]$ is strongly normalizing for every strongly normalizing v. Then $I(n, a; u.h)$ is strongly normalizing.*

Proof. Taking $v = u$ shows that h is strongly normalizing. Induct lexicographically on

$$(\nu(n),\ |n|,\ \nu(a) + \nu(h)).$$

It suffices to show that every one-step reduct of the iterator is strongly normalizing. A contraction inside n decreases the first coordinate. A contraction inside a or h decreases the last. If $h \to h'$, then $h'[v/u]$ is a reduct of $h[v/u]$ for every v, so the universal body hypothesis persists. The zero root reduct is a. For a successor root $n = Sp$, we have $\nu(p) = \nu(n)$ and $|p| < |n|$. The induction hypothesis makes $I(p, a; u.h)$ strongly normalizing, and the hypothesis about h then applies to the successor root reduct $h[I(p, a; u.h)/u]$. □

Theorem 3.2 (Strong normalization)**.** *Replacing the free variables of any fixed finite raw term by strongly normalizing terms produces a strongly normalizing term. In particular, every finite raw term is strongly normalizing.*

Proof. Induct on the fixed raw term, using simultaneous capture-free substitutions. Variables, zero, and successor are immediate. At an iterator, first rename its bound state variable away from the finitely many substituting terms. The structural induction hypotheses give strongly normalizing counter and initializer instances. The induction hypothesis for the strictly smaller update term applies to every additional strongly normalizing assignment for its state variable. It gives exactly the universal premise of Lemma 3.1. The identity substitution yields the last assertion. □

3.2 Confluence and normalization–substitution coherence

Lemma 3.3 (Confluence)**.** *The reduction* (Raw computation) *is confluent. Every term has a unique normal form $N(t)$, up to alpha-renaming, and literally with de Bruijn indices.*

Proof. First check local confluence. Disjoint redex contractions commute, and the zero and successor root patterns do not overlap. If an inner redex lies inside a root contraction, contract each of its residual copies on the other path. A zero contraction returns its initializer and erases the entire update; erased residuals require no step. A successor contraction may copy its recursive iterator into several update positions. The update is finite, so only finitely many copies arise, and substitution stability gives a common reduct after contracting them. An inner contraction of the update itself is handled both in the outer substituted update and in the retained recursive iterator. These are all nested overlaps.

Strong normalization and local confluence imply confluence: induct on the reduction height of a common source to join the first steps of two finite reductions and then their remaining tails. Thus two normal forms reachable from a term are equal up to bound-variable naming. □

A term is normal exactly when its children are normal and no iterator counter is headed by 0 or by S. Formula normalization changes only numerical terms; the equality and D constructors are preserved.

Lemma 3.4 (Coherence)**.** *For every finite raw term t and capture-free substitution σ,*

$$N(t[\sigma]) = N\big(N(t)[x \mapsto N(\sigma x)]\big). \qquad \text{(Coherence)}$$

The same equality holds for formulas. Normalized substitution consequently obeys identity, composition, renaming, and the usual binder-lifting laws.

Proof. The reduction $t \to^* N(t)$ survives raw substitution. Within the finitely many substituted copies, reduce each σx to $N(\sigma x)$. The resulting terms are convertible, and confluence makes their normal forms identical. Applying this observation twice and using composition of raw substitutions proves

$$N\big(N(t[\sigma])[\tau]\big) = N\big(t[x \mapsto N((\sigma x)[\tau])]\big).$$

Identity, renaming, and lifting are special cases. The formula assertion follows structurally because formula constructors do not reduce. □

The strict implementation also terminates. The selected traversal normalizes the counter, initializer, and update first. At a successor counter it normalizes the predecessor iterate before inserting it into the update. If the update ignores its state, that recursive value would have been erased by the direct raw root contraction, so this strict evaluation deserves an explicit justification.

Introduce an administrative term $\mathsf{let}\ u = a\ \mathsf{in}\ h$, with its usual capture-free reduction to $h[a/u]$, and replace the successor root rule in the auxiliary syntax by

$$I(Sn, a; u.h) \longrightarrow \mathsf{let}\ v = I(n, a; u.h)\ \mathsf{in}\ h[v/u], \qquad \text{(Strict expansion)}$$

where v is fresh. Reduce under all constructors and let components. The zero rule is unchanged. There are no new normal forms: every let has a root contraction, and its contraction composes a strict expansion to the original successor computation.

For this auxiliary reduction, a let is strongly normalizing whenever its value is strongly normalizing and its body sends every strongly normalizing substitution to a strongly normalizing term. Induction on the sum of the component reduction heights proves this: internal contractions decrease the sum, and the root contraction is covered by the universal body hypothesis. The proof of Lemma 3.1 now uses this let-closure fact at its successor root, after obtaining the predecessor iterate by the same lexicographic induction. Structural induction, also including the let constructor, therefore proves strong normalization of the auxiliary syntax.

The strict traversal is an evaluation strategy in this auxiliary calculus: after strict expansion, it first normalizes the let value and then contracts the let. Thus its recursive computational work cannot continue indefinitely, even in the erased-value case. Between those contractions it performs finite tree traversals and index renamings. Forgetting administrative lets preserves conversion in the original calculus; its returned normal term is therefore the unique $N(t)$. The same argument and (Coherence) identify hereditary substitution with normalized capture-free substitution. No uniform termination theorem inside iPRA has been invoked.

3.3 The raw reference calculus and its harmless clause convention

Let $\mathcal{C}_{\mathrm{raw}}$ be raw Choice-DI over the terms of (Raw computation) and formulas $s = t, D_{x,y}B$. It has the two raw computation equations, formula transport, clause Use, ordinary paired elimination, and the two separate introductions

$$\frac{\Gamma, B(t,0) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y}B}, \qquad \frac{\Gamma, E(t,z) \vdash E(t,Sz)}{\Gamma \vdash D_{x,y}E}, \qquad \text{(Raw introductions)}$$

where the second matrix is a literal equation and all eigenvariable conditions are as in the inherited Choice-DI foundations.

For comparison we allow clauses with zero or one antecedent and retain $\mathcal{R} = [\,; y.y = y]$. This is only an interface convention. In the inherited single-antecedent raw calculus, replace an unconditional clause $[\,; y.B]$ by $[H; y.B]$ and supply the theorem H to each use. Every use of $\mathcal{R}$ is replaced by derived raw reflexivity. Conversely the optional-premise interface admits all single-antecedent clauses. The inherited computation theorem used as a neutral antecedent there and H are both theorems, so those ordinary-assumption encodings are interchangeable by inserting the corresponding neutral proof. No new arithmetical or logical axiom is added by this convention.

The included Choice-DI development derives equation induction with arbitrary passive contexts, numerical cases, all ordinary equality reasoning, and binder congruence. Its logical and scalar arithmetic bridges identify its ordinary realization with full first-order iPRA. We use those results, rather than an assertion only about standard natural-number models.

3.4 Raw provability of normalization, including update binders

Transport in a numerical parameter does not permit a free variable in an equality proof to become captured by an iterator update. The needed update congruence follows from literal-equation induction.

Lemma 3.5 (Update extensionality)**.** *If u is fresh for Γ and $\Gamma \vdash_{\mathcal{C}_{\mathrm{raw}}} h(u) = k(u)$, then, after freshening binders away from the passive data,*

$$\Gamma \vdash_{\mathcal{C}_{\mathrm{raw}}} I(n, a; u.h) = I(n, a; u.k)$$

for every counter n and initializer a.

Proof. Apply derived literal-equation induction to $E(z) := (I(z, a; u.h) = I(z, a; u.k))$, with z fresh for the retained data. At zero both sides compute to a. In the successor step, write the left and right predecessor iterates as L and R. Computation gives $h(L)$ and $k(R)$. Capture-free numerical substitution in the supplied pointwise proof gives $h(L) = k(L)$; the induction hypothesis $L = R$ and ordinary term transport give $k(L) = k(R)$. Transitivity and the two successor computations prove $E(Sz)$. Induction gives $E(n)$. Only the literal equation E is inducted on. □

Lemma 3.6 (Internal normalization equivalence)**.** *For every fixed finite raw term t,*

$$\vdash_{\mathcal{C}_{\mathrm{raw}}} t = N(t).$$

For every fixed raw formula A, the formulas A and $N(A)$ are interderivable, uniformly under additional retained contexts.

Proof. Each root contraction is one raw computation equation. Ordinary transport handles a contraction inside a successor, counter, or initializer. For a contraction beneath an update binder, use Lemma 3.5, repeating it for nested update contexts. Thus every step in a finite normalization sequence is internally a provable equality. Transitivity gives $t = N(t)$.

For formulas, induct externally on formula depth. At an equation, normalize its two terms using the proved term equalities. At $D_{x,y}B$, open the source formula at a fresh witness, introduce the target at that same witness by case introduction, convert the new zero base to the old one by the induction hypothesis, use the old clause at a fresh successor, and convert the result back. Reverse the conversions for the reverse implication. Substitution may create fresh redexes in both an instance of B and the corresponding instance of $N(B)$: apply the smaller-depth induction hypothesis to both instances and use coherence to identify their common normal form. These are uniform derivations under every passive context. The construction uses derived binder congruence; it never captures a free variable by an ordinary equality transport. □

Replacing a retained clause. If antecedents A, A' are uniformly interderivable and bodies $B(y), B'(y)$ are uniformly interderivable, then the clauses $[A; y.B]$ and $[A'; y.B']$ are interchangeable in derivations. Replace each use of the old clause by conversion to the new antecedent, use of the new clause, and conversion of its specialized conclusion. Induction on proof trees, with fresh local eigenparameters, justifies this under further retained clauses. The same statement holds for unconditional clauses, with no antecedent conversion. In particular, a raw context and its normalized context have the same consequences after the corresponding formula conversions. This supplies the context-conversion step explicitly, rather than presuming that ordinary-assumption cut alone rewrites a conditional-universal clause.

3.5 Repairing the omitted case introduction at witness zero

Write $\mathcal{A}$ for the deterministic three-family calculus specified in Section 2. Its previously derived ordinary elimination and numerical cases require no equality transport. For unused u, put $q_v F(v) = D_{u,v}F(v)$. For an equation $E = (s = t)$ define

$$G_v E = (C(v, 0, s) = C(v, 0, t)), \qquad L(E) = q_v G_v E,$$

with all terms normalized and all new variables fresh. Direct computation gives $G_0 E \equiv H$ and $G_{Sw} E \equiv E$.

Lemma 3.7 (Compound envelope)**.** *In $\mathcal{A}$, an equation E and its compound envelope $L(E)$ are interderivable under every retained context, without equality transport.*

Proof. Under E, introduce the unused witness of $q_v G_v E$ at 1. Its successor premise is literally E, so this is case introduction. Conversely, open $q_v G_v E$ by derived ordinary elimination. Its clause has antecedent H. Prove H by the fixed reflexivity clause and use the opened clause at 1 to obtain E. □

Lemma 3.8 (Unrestricted case introduction)**.** *The case introduction in* (Raw introductions)*, with normalized substitution throughout, is derivable in $\mathcal{A}$ for every witness and every matrix.*

Proof. Only a literal-equation matrix $E(x, y)$ at the literal witness zero is missing from the primitive case mode. Suppose $\Gamma, E(0, 0) \vdash E(0, Sz)$ is supplied. Put $A(x, y) = L(E(x, y))$ with suitably fresh envelope binders. Under $A(0, 0)$, unwrap its envelope, insert the supplied derivation by ordinary-assumption cut, and wrap the result. This gives $A(0, Sz)$. Since A has a D root, primitive Intro at zero now selects cases, and yields $D_{x,y}A(x, y)$.

Open it by ordinary elimination at a fresh c. Introduce $D_{x,y}E(x, y)$ at witness c. Its syntax is a parameter rather than literal zero, so Intro again selects cases. Under $E(c, 0)$, wrap the equation to $A(c, 0)$, use the opened clause at Sz, and unwrap $A(c, Sz)$ to $E(c, Sz)$. Closing Intro and ordinary elimination gives the desired formula. No claim that $c \neq 0$ is used. □

Lemma 3.9 (Equation progress at every witness)**.** *The equation-progress introduction in* (Raw introductions)*, with normalized substitution, is derivable in $\mathcal{A}$ for every witness.*

Proof. Given $\Gamma, E(t, z) \vdash E(t, Sz)$, use a fresh unused witness coordinate w. Primitive Intro at zero on the equation matrix $E(t, y)$ yields $D_{w,y}E(t, y)$ by progress. Under the prospective outer case assumption $E(t, 0)$, open this formula by ordinary elimination and use its clause at Sz. This provides the successor premise for unrestricted case introduction of $D_{x,y}E(x, y)$ at witness t, available by Lemma 3.8. The unused coordinate prevents the opened witness from replacing t. □

3.6 An auxiliary normal-form calculus

Let $\mathcal{C}^+_{\mathrm{nf}}$ use canonical terms and formulas, normalized substitution, optional-premise clauses and the fixed reflexivity clause. Its rules are Use, ordinary paired elimination, full formula transport, and the separate unrestricted case and literal-equation progress introductions. There is no separate computation axiom, since both computation equations normalize to reflexivity.

Proposition 3.10 (Canonical Choice-DI and the new calculus)**.** *The calculi $\mathcal{C}^+_{\mathrm{nf}}$ and $\mathcal{A}$ derive the same canonical sequents.*

Proof. Translate $\mathcal{C}^+_{\mathrm{nf}}$ to $\mathcal{A}$ by using its derived ordinary elimination, term and formula transport, and Lemmas 3.8–3.9. Use and the fixed reflexivity clause are shared.

Conversely each primitive Intro is one of the two available choices. For Name, ordinary elimination opens its witness clause. When the deterministic guard succeeds, Use at 1, supplied with reflexivity of zero, proves the selected equality; otherwise reflexivity proves $b = b$. Full formula transport gives the exported normalized formula, whose stipulated freedom from b permits ordinary elimination to close. All translations preserve the entire canonical end sequent. □

3.7 Both proof translations and both composites

Let T normalize all numerical terms in raw formulas and clauses. Let U regard canonical syntax as raw syntax. These are operations on finite descriptions, not extra operations on the numerical domain.

Theorem 3.11 (Direct raw/canonical comparison)**.** *For every raw sequent and every canonical sequent,*

$$\begin{aligned} \Gamma \vdash_{\mathcal{C}_{\mathrm{raw}}} A \quad &\Longleftrightarrow \quad T\Gamma \vdash_{\mathcal{A}} TA, \\ \Delta \vdash_{\mathcal{A}} B \quad &\Longleftrightarrow \quad U\Delta \vdash_{\mathcal{C}_{\mathrm{raw}}} UB. \end{aligned}$$

Moreover $TU(B) = B$ literally, and A and $UT(A)$ are uniformly interderivable in the raw calculus. The corresponding clause-context composites are interchangeable in every derivation.

Proof. First translate a raw derivation into $\mathcal{C}^{+}_{\mathrm{nf}}$ by induction on derivations. The two computation equations have identical normal forms on their two sides, so become reflexivity. Coherence (Coherence) makes the other normalized premises and conclusions instances of the corresponding canonical rules. Equation-root status is preserved, and normalization cannot introduce a new free eigenparameter. Crucially, the auxiliary calculus retains both introduction choices, so this translation never assumes that a nonzero-syntax raw witness stays nonzero after normalization. Proposition 3.10 then translates to $\mathcal{A}$.

In the reverse direction first translate $\mathcal{A}$ to $\mathcal{C}^{+}_{\mathrm{nf}}$. Regard its canonical formulas as raw. For a raw rule whose substitution instance is not already normal, Lemma 3.6 converts the normalized premises to raw instances, the raw rule is applied, and its conclusion is converted back. The retained clause of an elimination may have a normalized zero antecedent instead of the literal raw substitution instance; the explicit clause-replacement construction above converts that context. The same reasoning handles Intro's temporary ordinary assumption. Raw reflexivity supplies the fixed reflexivity-clause uses. Thus induction on canonical derivations gives a raw proof with the stated end sequent.

On canonical syntax TU is literal identity. On raw syntax UT is formula normalization, whose internal equivalence and clause interchangeability were proved above. Composing the two proof translations with these equivalences gives both reverse implications in the displayed statements as well as the composites. □

Corollary 3.12 (Full first-order bi-interpretation)**.** *Composing the direct comparison with the included Choice-DI logical and scalar arithmetic bridges gives a bi-interpretation of the present calculus with full first-order intuitionistic primitive recursive arithmetic. Both numerical domains are whole, both equalities are actual equality, and both numerical composite isomorphisms have graph $x = y$. Both complete formula composites are internally provable. The theories are equiconsistent.*

Proof. The inherited bridge reconstructs all intuitionistic logical connectives and both quantifiers, quantifier-free induction, and the fixed-description primitive-recursive function symbols, with both formula round trips. Theorem 3.11 supplies the remaining change of term presentation. Its operations commute with substitution modulo internally provable computation equalities, so it preserves those interpretations. Numerical variables normalize to themselves: the comparison changes finite term descriptions, not the numerical domain or equality. Hence the inherited numerical identity graphs remain $x = y$.

The common designated contradiction $0 = S0$ is already normal and is preserved in both directions. The full formula comparisons therefore imply equiconsistency, as well as the stronger bi-interpretability statement. □

3.8 Uniform normalization is external

The full first-order iPRA convention here has a symbol for each fixed primitive-recursive definition and the corresponding defining equations, with quantifier-free induction in intuitionistic logic. It is neither the equation-only fragment nor a theory with unrestricted formula induction. A fixed scalar term compiles to one such primitive-recursive definition; no numerical variable ranges over all definitions inside the object theory.

Uniform normalization over arbitrary finite scalar-term codes is total recursive but is not primitive recursive. Otherwise, by coding a unary scalar term, substituting a numeral, normalizing, and extracting the resulting numeral, one would obtain a primitive-recursive universal evaluator $V(e, n)$ for all primitive-recursive functions. Invalid codes could be assigned zero. The diagonal function $V(e, e) + 1$ would itself have a scalar-term code, contradicting universality. This uses the included scalar compilation theorem, not an assumed uniform evaluator.

The size of a complete checker must count its normalizer, and the unbounded interface has no primitive-recursive running-time bound. Explicit finite fuel or reduction traces give a different, primitive-recursive bounded certificate interface for the same theorems. They do not add an arithmetic assertion of a uniform bound on the necessary fuel. The metatheorems here are written proofs; the finite executable checks are implementation evidence rather than a new Lean or Rocq verification of them.

4 Complete checker, certificate syntax, and measurements

The trusted executable is one Python module, `code/checker.py`. It imports only `json` and `sys` from the standard library. Proof construction, translation, measurement, and testing are outside the acceptance kernel. The complete normalized module is printed in Appendix F. Standard Python and its JSON implementation are the common host environment, as in the repository's comparisons.

4.1 Numerical and formula codes

The integer 0 is numerical zero. Negative integers denote free parameter levels; positive integers denote bound de Bruijn indices, starting with 1 for the nearest binder. Successor is the two-entry array `[0,t]`, and iteration is `[n,a,h]`, binding one variable in `h` only. An equation is `[s,t]`; a paired formula is `[B]`, binding two variables in `B`. Sorting distinguishes the successor array from an equation. The walker checks the original syntax before returning its normalized form, so malformed material in an erased iterator body is not accepted.

At free scope m and binder depth d, an integer numerical leaf must satisfy $-m \leq i \leq d$. Boolean values are not integers for this test. An iterator adds one to d in its body; a D adds two. Opening the outer witness of D removes its index 2 while preserving the inner control index 1. Bound substitution shifts an inserted term under internal binders. In contrast, a normalized target obtained from the fully opened first clause instance has no loose positive indices; free-parameter replacement may insert it without an additional shift.

Contexts are Python lists of pairs `(base,body)`. An empty array `base=[]` means no antecedent. The initial context is `[([],[1,1])]`, the exact code of the fixed reflexivity clause. Each Intro or Name extends the context by one clause and introduces one numerical parameter. Thus a context of length n has free scope $n - 1$, and its next parameter is $-n$. No additional scope counter is stored.

4.2 A uniform three-field proof packet

The checker synthesizes the proved formula. Every used proof packet has three fields:

$$[\text{code}, \text{arg}, \text{branch}].$$

The three alternatives are disjoint.

Packet	Meaning	Supplied data
`[j,t,p]`	Use	clause index, target, antecedent proof
`[B,t,p]`	Intro	nonempty matrix, witness, branch proof
`[[],p,q]`	Name	major proof, branch proof

In Use, j is an exact nonnegative integer below the context length. If the selected clause has no antecedent, its third field is unused; `null` is the suggested value. This is an explicit certificate convention, shared with the control. Intro's nonempty first field is a matrix well scoped under the two binders of its resulting D formula. The empty array is not a formula and selects Name. The latter infers its matrix from its major proof and its local formula F from its branch. It has no separate matrix, formula, replacement-term, or equality-proof payload.

The command-line input has exactly two keys, `goal` and `proof`. The goal must be a closed canonical formula and must equal the inferred conclusion literally. Successful checking prints `ACCEPT` and exits zero; every caught failure prints `REJECT` and exits nonzero. The trusted implementation contains no `assert` statements, so `python -O` does not disable checks. The optional fuel interface is described separately below.

4.3 The accepted-trace invariant

For a well-formed context of length n, every successful call to `derive` returns a canonical formula scoped over parameters $-1, \ldots, -(n-1)$ and a valid derivation of that formula. The proof is induction on the successful finite call tree.

Use validates its target, checks its antecedent when present, and returns the normalized instance of the selected valid clause. Intro checks the successor branch under its constructed ordinary assumption and returns the paired matrix. The final syntax check validates that original matrix including material which normalization might otherwise erase. This late check is sufficient for the accepted-trace induction: once a call has succeeded, the branch's context is known to have been well formed.

At Name, unpacking the major's single child requires a D root. The zero instance supplies the antecedent of precisely the clause being opened. If that antecedent is literally $[0, 0]$, the matrix has equation root; numerical substitution and normalization preserve formula roots, so its first instance is also an equation. A separate length test for that row would therefore be redundant. The mapper rewrites the current fresh leaf only when it is also the first row's left side and the base is $[0, 0]$. This is exactly the deterministic test in (First-instance test). The final scope check rejects a witness which survives in the exported normal form. Inserted targets are not visited again, so the substitution is simultaneous.

The converse direction follows by encoding the three inference families recursively. Physical Python resource limits can reject a sufficiently large finite derivation; as usual, mathematical completeness concerns the specified finite-tree algorithm with adequate resources.

4.4 Measured comparisons and their interpretation

All reported normalized measurements use CPython 3.12.14: remove documentation strings, apply `ast.unparse`, and append one final line feed. Significant tokens exclude comments, indentation tokens, and line/end markers. AST nodes are counted by `ast.walk`. Gzip uses maximum level 9 and timestamp zero and is informational. Every acceptance helper, parser entry, scope and syntax check, normalization step, and final acceptance test in the module is counted.

Complete module	Bytes	Tokens	AST	Gzip-9
New deterministic Name, packets	2157	704	804	870
Matched unrestricted Name, packets	2228	701	813	894
Exact incoming witness discharge	2273	732	813	903
Exact incoming uniform binding	2238	693	764	866
New deterministic Name, with fuel	2340	752	863	936
Matched unrestricted Name, with fuel	2411	749	872	959
Existing native public-goal profile	788	358	413	460
Existing compact canonical profile	516	265	306	332

The matched packet control uses the same syntax, normalizer, validator, context representation, source conventions, and input/output contract. Only Name differs: it receives an equality branch as well as a formula branch. The new restriction saves 71 normalized bytes (3.19%), 9 AST nodes, and 24 gzip bytes; it costs 3 significant tokens. This is an actual checker-size reduction with an explicit coordinate tradeoff, not a claim of coordinatewise dominance over the matched control.

Against the exact incoming witness-discharge module, the new combination saves 116 normalized bytes (5.10%), 28 tokens, 9 AST nodes and 33 gzip bytes. That comparison includes the certificate-interface change and should not be attributed entirely to the mathematical premise deletion. Against the incoming uniform-binding module, it saves 81 bytes but has 11 more tokens and 40 more AST nodes. The two compact historical profiles remain substantially smaller; their source sizes are not new records of this package. No claim that this construction beats the repository's global lower envelope is justified by these data.

The measured raw readable source of the selected checker is 2544 bytes, or 1059 bytes with gzip-9. Complete source hashes and every normalized and compressed module are supplied with the measurements.

4.5 Executed evidence

The executable replay suite passed 16 positive cases and 57 adversarial rejection cases, with 178 standalone process checks across ordinary and optimized Python. Thirteen positive cases start from inherited proofs, pass through the separate-rule control, and are translated into the new calculus. All sixteen selected proofs are decoded, checked by an independent annotated reference implementation, and translated back to the old witness-discharge checker with exactly the same end formula.

The positive cases include symmetry from a retained assumption, transitivity using both equality premises, repeated and nested term occurrences, substitution below formula binders, equation induction that uses its induction hypothesis, and the zero-witness case repair that uses its base. New boundary cases exercise default discharge, the literal-H restriction, and an active self-containing replacement whose fresh occurrences disappear after normalization. Rejection cases cover the wrong equation source, an unsuitable base, escaped witnesses, malformed erased syntax, ill-scoped terms, wrong JSON types, and malformed proof packets. The machine-readable log identifies individual cases.

These checks provide executable evidence for the implementation and translations. The universal soundness, completeness, normalization, and bi-interpretation claims are the written mathematical theorems in this report. This delivery does not claim a new Lean or Rocq verification.

5 A primitive-recursive certificate interface

Full normalization is a terminating operation on well-formed scalar-iterator terms, but its unbounded running time should not be confused with a primitive-recursive bound uniform

in arbitrary input descriptions. The optional bounded checker makes the certificate relation primitive recursive without requiring such a normalization bound.

Exact interface. The input object has exactly three fields, `goal`, `proof`, and `fuel`. The first two have the same meanings as in the unbounded checker. The last is an exact nonnegative integer; in particular, Boolean values are rejected rather than treated as the integers zero and one. The budget is reset at the start of each call to `check`. Every entry to `walk` or `derive` first requires a positive remaining budget and then subtracts one. Exhaustion rejects the certificate. No other mathematical rule, term operation, scope check, or normalization convention changes. The same transformation is applied to the matched old-discharge control.

The budget counts recursive entries, not seconds, bytes, reductions, or Python instructions. A successful run may leave unused fuel. This explicit interpretation is necessary when comparing the new and control checkers: one may perform more normalization traversals even when its proof interface contains fewer premises.

Soundness and preservation of the theorem set. Erase the calls that decrement and test the budget. Every accepting bounded run then becomes an accepting run of the corresponding unbounded checker on the same goal and proof. Thus adding fuel cannot introduce a theorem. Conversely, consider any particular successful finite run of the unbounded checker. Let k be the number of entries to its two metered procedures. A budget of exactly k follows that same run and accepts; every larger budget does so as well. A budget of $k-1$ rejects at the last required entry. Hence existentially quantifying the fuel field preserves the accepted theorems:

$$\exists p\ \mathrm{Accept}(A,p) \quad\Longleftrightarrow\quad \exists p\,\exists k\ \mathrm{Accept}_{\mathrm{fuel}}(A,p,k).$$

This argument concerns each fixed finite derivation. It does not assert a primitive-recursive bound on the fuel needed by all proofs, nor does it assert that normalization itself has such a uniform bound.

Why bounded acceptance is primitive recursive. Use an ordinary finite-tree encoding of parsed input data, stack frames, closures from the fixed program, and finite lists. Parsing finite JSON and the initial exact-key and integer tests can be implemented by bounded search and primitive recursion on the input string.

Every cycle in the checker call graph passes through `walk` or `derive`. The other helpers either perform a finite local operation or immediately enter one of these procedures. The callbacks supplied to `walk` come from a fixed finite collection of program expressions; the certificate cannot supply executable callbacks. In particular, a shift requested by a callback enters `walk` and consumes fuel. A run with initial fuel k therefore has at most k successful metered activations, with an additional failing entry possible on exhaustion. Unwinding those activations cannot restart an unmetered recursive computation.

The remaining local operations are integer arithmetic and comparison, finite list construction and access, list or tree equality, and traversal of a finite input collection. Each has a primitive-recursive implementation. Tree sizes do require an explicit bound, because substitution can duplicate an existing subtree. Such duplication does not invalidate primitive recursiveness: a single local constructor or substitution-return operation has output length bounded by a fixed polynomial in the total length of the already available finite data. List comparisons inspect only those data. The number of stack-frame entries and returns is bounded by a constant multiple of $k+1$, and all completed iterations of an explicit traversal either inspect input data or enter a metered child.

More conservatively, let N bound the length of the input encoding and let $b_0 = N + k + 10$. Iterating an elementary majorant, for example

$$b_{j+1} = 2^{(b_j+1)^2},$$

a fixed constant multiple of $k + N + 1$ times bounds the finite data and continuations generated by this fixed program. The constant is chosen larger than its fixed number of local operations per activation; it does not depend on a certificate. A further elementary bound covers the local list comparisons and integer operations. The resulting total bound is primitive recursive in (N, k), by primitive recursion on the displayed iteration count. One may consequently simulate the defunctionalized checker for that many elementary steps and return its acceptance bit. This establishes a primitive-recursive predicate for bounded acceptance.

The bound is deliberately very loose. No such majorant is evaluated by the delivered Python checker: it simply maintains the integer budget. The argument explains why its abstract bounded execution has a primitive-recursive arithmetization, rather than assuming that bounded normalization is free.

Implementation boundary. As with the unbounded reference implementation, a concrete Python process can reject a certificate because of a finite stack, memory, parser, or execution limit. The finite-derivation completeness assertion above uses the abstract algorithm with adequate resources. Such implementation limits can only add rejection; they do not supply an additional inference rule. The fuel field is an explicit mathematical bound and is independent of those practical limits.

Executed boundary checks. The accompanying validation script tests five accepted certificates for both the new and matched control checkers: closed reflexivity, a default discharge with the wrong row source, an active self-containing target whose occurrences disappear after normalization, induction that uses its hypothesis, and substitution beneath a formula binder. For each checker and certificate it measures the exact metered cost, verifies acceptance at that cost, verifies rejection at one less and at zero, rejects eight malformed budget values, and checks the exact object-key contract. The exact-budget and one-less outcomes are also tested through the standalone command-line entry point. The machine-readable results and the full-module size measurements are in `fuel_verification.json`.

Attribution and role of the inherited foundations

The following five sections reproduce the mathematical reference fragments from the pinned `certified_zero` package [1]. They retain their original attributions and theorem dependencies. Words such as "new" and "old" within those inherited fragments refer to their original comparison, not to the present first-instance rule. The raw clause convention in the first fragment represents an ordinary assumption through a fixed true antecedent; an unconditional constant clause gives exactly the same assumption, since that antecedent is a theorem. The canonical comparison in Section 3 supplies the passage to the present syntax and initial-clause convention.

Their purpose is substantive: they derive the ordinary logical rules, coverage, predecessor, reflection of actual equality, quantifier-free induction, pairing, and primitive recursion, and prove both numerical and formula composites internally. The new local theorem depends on that bridge and does not claim to replace its arithmetic construction.

A Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

A.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)}\ \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0,1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

A.2 Exact equivalence and the unused-witness construction

Lemma A.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem A.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y} E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition A.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

A.3 Induction and numerical cases separate cleanly

Lemma A.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma A.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0,t;u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

A.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \qquad \text{(Gate and padding)}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B), \\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B), \\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A, B), \\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A, B), \\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x), \\ A \vee_* B &:= \exists_* x\, M_x(A, B), \\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z), \\ \bot_* &:= (0 = 1). \end{aligned} \qquad \text{(Logical macros)}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every

control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

A.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$

$$\langle A; y.B\rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y} B \dashv\vdash \exists_* x \big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a, 0)$ that clause supplies $B(a, y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* y B$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem A.2 preserves every native sequent.

A.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

B Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x(B^\circ(x, 0) \to \forall y B^\circ(x, y)).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section A recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and

return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

B.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.

2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.

3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.

4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n, \vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

Equation induction proves $L(V(n, \vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem B.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros

and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

C Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [5] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

C.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

C.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n) + 1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n + 2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

C.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m + n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

C.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \tag{12}$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \tag{13}$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \tag{14}$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \tag{15}$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

C.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

D From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [6]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix C, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

D.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$
$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{17}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x\,(x = 0 \vee x = S(Px)), \tag{18}$$
$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p}\,\left[(E(0, \vec{p}) \wedge \forall n(E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n\, E(n, \vec{p})\right]. \tag{20}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0, \vec{p}), \qquad \Gamma, E(z, \vec{p}) \vdash E(Sz, \vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n, \vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

D.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u, 0, 1), \qquad K(u) = C(u, 1, Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all u, v,

$$C(u, v, v) = v. \tag{27}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

D.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$

$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$

$$(Sx) + y = S(x + y), \tag{32}$$

$$d(Sx, Sy) = d(x, y), \tag{33}$$

$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S((Sx) + y) = S(S(x + y)) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

D.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C\big(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)\big) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at y and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a,b) = \mathrm{sg}(K(d(Sa,b))). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a,b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$
$$e(a,b) = 0 \ \vee \ e(a,b) = 1. \tag{40}$$

In detail, $e(a,b) = 0$ is equivalent to $K(d(Sa,b)) = 0$ by (25), which is equivalent to $d(Sa,b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa,b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a,b) = 0$ branch, (39) gives $a = b$. In the $e(a,b) = 1$ branch, an assumption $a = b$ would give $e(a,b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a\, \forall b\, (a = b \vee (a = b \to \bot)) \tag{41}$$

is a theorem, not an extra hypothesis.

D.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s,t), \qquad b_\bot = 1, \tag{42}$$
$$b_{A\wedge D} = C(b_A, b_D, 1), \tag{43}$$
$$b_{A\vee D} = C(b_A, 0, b_D), \tag{44}$$
$$b_{A\to D} = C(b_A, b_D, 0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$(b_A = 0 \vee b_A = 1) \quad \text{and} \quad (b_A = 0 \longleftrightarrow A). \tag{46}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (39)–(40). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

D.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

E Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [7]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

E.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a,b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \qquad \text{(Root invariant)}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n)+1)^2. \qquad \text{(Root)}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a,b) = (a+b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a+b$, one has $k^2 \le k^2 + a < (k+1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a,b)) = a+b$, and therefore

$$L(\pi(a,b)) = a, \qquad H(\pi(a,b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

E.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$

$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

E.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section A give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

F Complete normalized acceptance checker

The following is the complete measured module. Documentation and comments are removed by the measurement convention; every executable definition and the command-line entry point are present. The readable source and the optional bounded version are separate files in the archive.

```
import json
import sys

def need(condition):
    if not condition:
        raise ValueError('invalid certificate')

def walk(a, f, d=0, form=None):
    if type(a) is int:
        need(not form)
        return f(a, d)
    need(type(a) is list)
    size = len(a)
    need(form is None or (size in ((1, 2) if form else (2, 3)) and (form or size == 3 or a[0] ==
         0)))
    a = [walk(b, f, d + (2 if size == 1 else int(size == 3 and i == 2)), size == 1) for i, b in
         enumerate(a)]
    if size == 3:
        c, b, h = a
        if c == 0:
            return b
        if type(c) is list and len(c) == 2:
            return sub(h, walk([c[1], b, h], lambda i, d: i))
    return a

def syntax(a, n, form=False):

    def variable(i, d):
        need(-n <= i <= d)
        return i
    need(walk(a, variable, form=form) == a)

def shift(a, k):
    return walk(a, lambda i, d: i + k if i > d else i)

def sub(a, t, depth=0):
    return walk(a, lambda i, d: shift(t, d) if i == d + 1 else i - (i > d + 1), depth)

def derive(packet, g):
    n = len(g)
    code, arg, branch = packet
    if type(code) is int:
        need(0 <= code < len(g))
        syntax(arg, n - 1)
        base, body = g[code]
        if base:
            need(derive(branch, g) == base)
        a = sub(body, arg)
    elif code:
        syntax(arg, n - 1)
        body = sub(code, arg, 1)
        hyp = sub(body, -n if len(body) == 2 and arg == 0 else 0)
        need(derive(branch, g + [([], hyp)]) == sub(body, [0, -n]))
        a = [code]
    else:
        need(code == [])
        body, = derive(arg, g)
```

```
        body = sub(body, -n, 1)
        base = sub(body, 0)
        row = sub(body, [0, 0])
        a = walk(derive(branch, g + [(base, body)]), lambda i, d: row[1] if base == [0, 0] and
            row[0] == i == -n else i)
    syntax(a, n - 1, True)
    return a

def check(data):
    need(type(data) is dict and set(data) == {'goal', 'proof'})
    syntax(data['goal'], 0, True)
    need(derive(data['proof'], [([], [1, 1])]) == data['goal'])
if __name__ == '__main__':
    try:
        check(json.load(sys.stdin))
    except Exception:
        print('REJECT')
        sys.exit(1)
    print('ACCEPT')
```

G Reproduction and file map

With Python 3.12.14, run the following from the extracted directory:

```
python3 reproduce.py
python3 code/checker.py < tests/positive/successor_congruence.json
python3 code/checker.py < tests/negative/goal_false.json
python3 build_pdf.py
```

The first selected example prints `ACCEPT`; the second prints `REJECT`. The reproduction entry point verifies the delivered manifest, replays positive and negative certificates and proof translations, checks the bounded variants, and recomputes complete-module measurements. The PDF build uses an installed pdfLaTeX distribution with shell escape disabled. A freshly compiled PDF may differ in metadata and therefore in its file hash; the delivered manifest describes the original payload.

File or directory	Contents
`ipra_first_instance.tex`	Standalone LaTeX source containing this entire report.
`ipra_first_instance.pdf`	Compiled report.
`paper/`	Editable source sections used to assemble the standalone source.
`code/`	Complete selected and matched control checkers, with and without fuel.
`tests/`	Positive, negative and source certificates; untrusted proof constructors, translators and independent replay.
`reference/`	Attributed foundation fragments and exact comparison checker snapshots.
`evidence/`	Measurements, normalized and gzip sources, verification logs and provenance.
`FORMAT.txt`	Public JSON grammar and scope conventions.
`CLAIMS.json`	Explicit positive claims and limitations.
`SOURCE_PROVENANCE.json`	Pinned repository and mathematical source identities.
`MANIFEST.json`	SHA-256 checksums of every other payload file.

References

[1] Vladimir Reshetnikov, *ProveIt: Concise presentations of intuitionistic primitive recursive arithmetic*, snapshot `17898ea7e6a1300dfdcad71040d5daeed67916d4`, `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`. https://github.com/VladimirReshetnikov/ProveIt/tree/17898ea7e6a1300dfdcad71040d5daeed67916d4/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA.

[2] *iPRA with equality at witness discharge*, `docs/incoming/ipra_witness_discharge_2026-11-11.zip` in [1], especially the complete native specification, naming-matrix proof, zero-witness repair, and normalization appendix. The archive's own original snapshot is `0ebd0c6b358f38bb0ad2883faf775421599c8aae`.

[3] *Full normalization*, `Research/full_normalization` in [1]; normalization results as reproduced and attributed in [2]. The present bridge additionally spells out a termination justification for strict normalization of an unused predecessor iterate.

[4] *Uniform binding for intuitionistic primitive recursive arithmetic*, `docs/incoming/ipra_uniform_binding_2026-11-11.zip` in [1]. Exact checker source retained for comparison.

[5] *Internal scalar predecessor bootstrap*, the attributed development in `Research/certified_zero/reference/scalar_predecessor.tex` of [1]; reproduced here without a claim of new authorship.

[6] *From literal-equation induction to full intuitionistic PRA*, the attributed forward-left development in `Research/certified_zero/reference/equality_reflection.tex` of [1]; reproduced here.

[7] *Pairing, primitive recursion, and internal numerical composites*, the attributed scalar arithmetic development in `Research/certified_zero/reference/primitive_recursion.tex` of [1]; reproduced here.

[8] Erfan Khaniki, *Not all Kripke models of HA are locally PA*, *Advances in Mathematics* 397 (2022), 108126. Author manuscript dated 9 January 2021, Section 2.1. https://iuuk.mff.cuni.cz/~koucky/EPAC/papers/HA_Final.pdf. Used for the full first-order iPRA convention, not as a source of the new inference restriction.